Moulinette Lab

GUIDE UTILISATEUR · VERSION 0.29-DEV

Apprendre, tester et comprendre ses corrections

Un manuel pour installer et utiliser le prototype local, lire les résultats et suivre sa progression sur les exercices C de la Piscine.

À retenir : projet pédagogique non officiel, sans affiliation à 42. Les tests ne sont pas ceux de 42 et un résultat réussi ne garantit pas une conformité complète.

Sécurité : la protection n'est pas achevée. Utilisez uniquement du code et des dépôts de confiance, que vous avez écrits ou examinés. Aucun mode n'est conçu pour exécuter du code hostile.

Dans ce guide, les commandes supposent Ubuntu et un terminal ouvert dans le dossier du projet extrait. Les noms de chemins exemples sont à adapter à votre ordinateur.

01 · Présentation et parcours

Qu'est-ce que Moulinette Lab ?

Moulinette Lab est un correcteur pédagogique local pour le profil 42 autodidacte. Il compile un rendu de confiance, effectue des contrôles et des tests fonctionnels, puis présente un rapport. Un tableau de bord HTML permet de consulter l'historique enregistré sur l'ordinateur. Il fonctionne hors ligne pour les rendus locaux ; récupérer un dépôt GitHub nécessite le réseau.

Les 14 sujets C Piscine fournis ont été analysés et intégrés manuellement. Le catalogue comprend **109 exercices**, de C00 à C13. Selon le sujet, un rendu peut comprendre un fichier C, plusieurs sources, des en-têtes, un Makefile, un script, une bibliothèque ou un programme.

Contenu du guide

- 02 · Installation et prérequis
- 03 · Première correction et utilisation quotidienne
- 04 · Comprendre les résultats et les modes d'exécution
- 05 · Corriger un dépôt GitHub de confiance
- 06 · Tableau de bord, thèmes et historique
- 07 · Référence des commandes et options
- 08 · Dépannage et messages courants
- 09 · Exercices pris en charge et limites pédagogiques
- 10 · Données, sécurité et état du projet

Ce que le Lab ne fait pas

Il ne prétend pas reproduire la moulinette officielle ni sa notation. Il ne télécharge pas automatiquement des sujets et ne crée pas automatiquement des tests à partir d'un PDF ou d'un Markdown. Les sujets de la Piscine eux-mêmes ne sont pas fournis dans l'archive ; référez-vous aux documents que vous possédez.

02 · Installation

1. Préparer les outils système

La version actuelle vise Ubuntu avec Python 3.10 ou plus, un compilateur C (`cc`), `make`, `ar`, `nm` et Bubblewrap. Git est nécessaire pour les dépôts GitHub. Si les outils sont déjà installés, inutile de les réinstaller.

Sur une installation Ubuntu où ils manquent, on peut installer les paquets suivants. Cette étape demande les droits d'administration et une connexion :

```
sudo apt update
sudo apt install python3 python3-venv build-essential binutils \
  make bubblewrap git unzip
```

2. Préparer la Norminette

La version attendue est **Norminette 3.3.59**, disponible dans le `PATH` et installée dans un environnement virtuel détectable. Vérifiez d'abord votre installation existante :

```
command -v norminette
norminette --version
command -v bwrap
```

Si la Norminette est absente, vous pouvez créer un environnement dédié et l'activer :

```
python3 -m venv ~/.venvs/moulinette-norm
source ~/.venvs/moulinette-norm/bin/activate
python -m pip install 'norminette==3.3.59'
norminette --version
```

Dans un nouveau terminal, réactivez cet environnement avec la commande `source` ci-dessus si la Norminette n'est plus trouvée. Si votre installation actuelle fonctionne déjà, conservez-la.

02 · Installation, suite

3. Extraire le Lab

Téléchargez l'archive v0.29-dev et extrayez-la dans un **nouveau dossier**, sans écraser une ancienne version ou vos rendus. Par exemple, si l'archive est dans Téléchargements :

```
cd ~/Téléchargements
unzip moulinette-lab-v0.29-dev.zip
cd moulinette-lab-v0.29-dev
python3 -c "import moulinette; print(moulinette.__version__)"
```

La version affichée doit être `0.29.dev0`. Vous pouvez aussi extraire l'archive avec le gestionnaire de fichiers, puis ouvrir un terminal dans le dossier extrait. Les commandes `python3 -m moulinette` de ce guide se lancent depuis cette racine tant que le paquet n'est pas installé.

Installation de la commande (facultative)

Il n'est pas nécessaire d'installer le Lab pour l'utiliser depuis le dossier extrait. Si vous souhaitez disposer de la commande `moulinette` dans un environnement virtuel, activez un environnement approprié, puis, depuis la racine du projet :

```
python -m pip install .
moulinette --help
```

La Norminette 3.3.59 doit aussi rester accessible dans le `PATH`. L'installation peut nécessiter des outils de construction et un accès réseau. Pour sortir d'un environnement virtuel : `deactivate`.

Organisation du dossier

- `moulinette/` : moteur, commandes et ressources du tableau de bord.
- `exemples/` : rendus de référence de confiance. C00 utilise des dossiers nommés par fonction ; les autres sont rangés par `cXX/exYY`.
- `tests/` : suite automatisée standard. `tests_real/` contient des essais séparés de scope systemd réel.
- `README.md`, `ETAT-CHANTIER.md` et `docs/` : documentation et historique. Ce guide est aussi dans `docs/`.

03 · Première utilisation

Vérifier un exemple fourni

Depuis le dossier du projet, lancez un exemple fixe de confiance avec Bubblewrap :

```
python3 -m moulinette check ./exemples/ft_putchar \
  --exercise c00/ex00 --experimental-sandbox
```

Le rapport liste les contrôles effectués et se termine par `Résultat : réussi`, `échec` ou `incomplet`. Le premier succès signifie uniquement que **les contrôles exécutés** ont réussi.

Un exemple multi-fichiers, puis un exercice de listes :

```
python3 -m moulinette check ./exemples/c10/ex02 \
  --exercise c10/ex02 --experimental-sandbox
python3 -m moulinette check ./exemples/c12/ex08 \
  --exercise c12/ex08 --experimental-sandbox
```

Corriger son propre travail

Remplacez le chemin par le dossier contenant les fichiers de votre rendu. Choisissez l'identifiant correspondant au sujet, par exemple :

```
python3 -m moulinette check /chemin/vers/mon/ex00 \
  --exercise c00/ex00 --experimental-sandbox
```

Le chemin est un **dossier**, pas directement le fichier C. Pour les exercices à fichiers imposés, ne laissez pas d'autres fichiers dans le dossier de rendu. Pour un projet à fichiers libres, respectez les fichiers et règles exigés par son sujet. N'utilisez que des fichiers de confiance, y compris les scripts et Makefiles qui peuvent être exécutés.

Ouvrir le tableau de bord

```
python3 -m moulinette dashboard
```

Le Lab génère une page HTML locale et tente de l'ouvrir. Après une nouvelle correction, relancez cette commande : la page est **statique** et ne se met pas à jour seule. Vous pouvez ensuite filtrer les exercices et ouvrir les détails des corrections.

04 · Résultats et modes

Lire le rapport

- **Réussi** : les vérifications exécutées ont réussi. Ce n'est pas une garantie de réussite aux tests officiels.
- **Échec** : au moins une vérification a trouvé une erreur. Lisez son nom et son message ; une étape échouée peut empêcher l'exécution des étapes suivantes.
- **Incomplet** : un outil ou un contrôle est indisponible, ou son résultat est ambigu. Il ne faut pas interpréter ce verdict comme une réussite.
- Les lignes d'information expliquent notamment les adaptations et limites ; elles ne sont ni une réussite ni un échec.

Pour `check`, les codes de sortie sont `0` (réussi), `1` (échec) et `2` (incomplet ou erreur d'utilisation). Un script shell peut examiner `$?` immédiatement après la commande.

Choisir un mode : un seul à la fois

- `--experimental-sandbox` : exécution expérimentale avec Bubblewrap, uniquement pour du code de confiance. En cas d'indisponibilité, pas de repli automatique sur le mode local.
- `--experimental-cgroup-sandbox` : Bubblewrap et scope systemd utilisateur avec limites expérimentales, si disponibles. Quatre exemples de confiance ont réussi en scope réel sur l'Ubuntu utilisateur en v0.28.3, dont C12/ex08 ; cela ne valide ni cette version ultérieure, ni les limites sous charge, ni la sécurité.
- `--unsafe-local` : exécution **sans isolation**, avec vos droits et l'accès de votre session. Réservée au code de confiance ; à éviter si vous ne comprenez pas ses conséquences.

Aucun mode n'est autorisé pour un dépôt ou un programme hostile. Le noyau est partagé ; il n'y a pas de garantie de confinement ni de filtre seccomp complet. Ne modifiez pas les protections du système pour forcer le fonctionnement.

Norminette adaptée

La Norminette 3.3.59 est appliquée au profil autodidacte : **seul** `INVALID_HEADER` **est ignoré**. L'en-tête institutionnel et les obligations intranet ne sont pas exigés. Les autres diagnostics restent visibles. C09/ex01 ne rend qu'un Makefile : la Norminette n'y est pas applicable. Pour les projets, le style est vérifié séparément sur la copie initiale du rendu, et non sur les fichiers que le Makefile aurait modifiés. En mode Bubblewrap, l'environnement Norminette n'est plus monté explicitement pendant la compilation ou l'exécution du projet. Cette séparation n'autorise pas le code non fiable.

05 · GitHub de confiance

Le Lab peut récupérer un dépôt GitHub **public en HTTPS** dont vous avez vérifié le contenu. Le clonage utilise le réseau de l'hôte via Bubblewrap, puis la correction utilise le mode choisi. Ce parcours est expérimental et ne rend pas un dépôt inconnu sûr.

```
python3 -m moulinette check \
  https://github.com/PROPRIETAIRE/DEPOT \
  --exercise c07/ex04 --subdir c07/ex04 \
  --trust-github --experimental-sandbox
```

Remplacez l'URL, l'identifiant et le sous-dossier par ceux de votre dépôt de confiance. L'option `--trust-github` est obligatoire pour confirmer votre choix. Sans `--subdir`, le dossier évalué est la racine du dépôt. Le chemin du sous-dossier doit être relatif, sans `..` ni liens symboliques dans ses composants.

Choisir une révision

Ajoutez `--ref main`, `--ref v1.0` ou `--ref` suivi d'un SHA complet de 40 caractères. Sans option, le `HEAD` par défaut du dépôt est utilisé. Les SHA abrégés ne sont pas acceptés. Le rapport contient le SHA réellement évalué et le sous-dossier.

```
python3 -m moulinette check \
  https://github.com/PROPRIETAIRE/DEPOT \
  --exercise c07/ex04 --subdir c07/ex04 --ref main \
  --trust-github --experimental-sandbox
```

La récupération utilise un espace temporaire Bubblewrap limité, mais ce n'est pas une protection garantie contre un dépôt hostile. Les redirections HTTP(S), sous-modules, dépôts privés, SSH et Git LFS ne sont pas pris en charge. Une erreur de réseau ou de récupération donne un résultat incomplet, sans repli local automatique. Le Lab travaille sur une copie temporaire ; il n'écrit pas dans le dépôt d'origine.

06 · Tableau de bord et historique

Suivre sa progression

Le tableau de bord affiche les **109 exercices**, leur état à la dernière tentative, le nombre de corrections et les **50 dernières corrections**. Une recherche par identifiant ou nom de fichier et un filtre par module sont disponibles. Cliquez sur une correction pour ouvrir ses détails.

- Vert « Réussi » : dernière tentative réussie.
- Rouge « À revoir » : dernière tentative en échec.
- Ambre « Incomplet » : résultat non concluant.
- Gris « À découvrir » : aucune tentative enregistrée.

Les libellés complètent les couleurs. Les données de l'aperçu de démonstration, s'il est utilisé, ne sont pas votre historique.

Thèmes et humour

Choisissez `Aurore`, `Night Owl`, `Océan`, `Forêt` ou `Braise`. Le thème est conservé par le stockage local du navigateur. `Aléatoire à l'ouverture` choisit un autre thème à chaque ouverture ou rechargement lorsque le stockage est accessible. Si le navigateur bloque ce stockage, la page le signale ; la préférence ne peut alors pas être conservée. Elle dépend du navigateur et du chemin de la page. Le bouton d'humour permet de masquer la case et mémorise aussi son choix lorsque le stockage est disponible.

Scores et données

Le score affiché est le nombre de contrôles réussis divisé par le nombre de contrôles réussis et échoués effectivement exécutés. Les lignes d'information sont exclues. Si un contrôle est indisponible ou que le résultat est incomplet, le score est masqué. Ce n'est **pas une note officielle**, et les scores ne constituent pas un barème comparable garanti entre exercices.

Par défaut, la base est `~/.local/share/moulinette-lab/history.sqlite3` et la page `~/.local/share/moulinette-lab/dashboard.html`. Si `XDG_DATA_HOME` est un chemin absolu, le dossier est placé sous celui-ci. Les diagnostics peuvent contenir du code ou des chemins : vérifiez-les avant de partager la base. `--no-history` évite l'enregistrement d'une correction donnée.

07 · Toutes les commandes principales

Les commandes ci-dessous se lancent depuis la racine du projet extrait. Ajoutez `--help` à une commande pour obtenir son aide ; seuls les modes de `check` sont mutuellement exclusifs.

Aide et version

```
python3 -m moulinette --help
python3 -m moulinette check --help
python3 -m moulinette dashboard --help
python3 -m moulinette sandbox --help
python3 -c "import moulinette; print(moulinette.__version__)"
```

Correction et options

```
python3 -m moulinette check /chemin/vers/rendu \
  --exercise c00/ex00 --experimental-sandbox
```

- `PATH` : premier argument obligatoire ; dossier local ou URL publique GitHub de confiance.
- `--exercise cXX/exYY` : identifiant de l'exercice, obligatoire.
- `--experimental-sandbox`, `--experimental-cgroup-sandbox` ou `--unsafe-local` : choisir explicitement **un seul** mode. Voir la section 04.
- `--json` : afficher le rapport sous forme JSON.
- `--no-history` : ne pas enregistrer cette correction.
- `--trust-github` : confirmer le dépôt GitHub public de confiance ; réservé aux URL.
- `--ref` : branche, tag ou SHA Git complet ; réservé à GitHub.
- `--subdir` : sous-dossier relatif du rendu ; réservé à GitHub.

Exemple de JSON sans enregistrement :

```
python3 -m moulinette check ./exemples/ft_putchar \
  --exercise c00/ex00 --experimental-sandbox \
  --json --no-history
```

07 · Commandes, suite

Tableau de bord

```
python3 -m moulinette dashboard
python3 -m moulinette dashboard --no-open
```

La première commande génère puis tente d'ouvrir la page. La seconde la génère et affiche son chemin, sans ouvrir de navigateur. Les boutons du tableau de bord téléchargent le manuel utilisateur et le guide du style C distinct ; ce sont des PDF statiques, pas des bilans personnalisés. Les détails de correction peuvent proposer des explications indicatives et comparer les contrôles avec la tentative précédente du même exercice parmi les 50 affichées.

Diagnostics expérimentaux

```
python3 -m moulinette sandbox doctor
python3 -m moulinette sandbox selftest
python3 -m moulinette sandbox cgroup-selftest
```

`doctor` examine les prérequis ; `selftest` compile et exécute des témoins fixes de confiance dans Bubblewrap ; `cgroup-selftest` utilise un scope systemd utilisateur et des témoins limités ; il atteint volontairement une petite limite de tâches et ne fait pas partie de la sélection sans charge. Ces diagnostics sont facultatifs, ne prouvent pas la sécurité et peuvent rendre un statut incomplet même si leurs vérifications visibles réussissent. Chacun accepte `--json`. Ne lancez jamais le témoin OOM de l'ancienne v0.20.

Tester le Lab lui-même

```
python3 tools/test_sans_charge.py
```

La sélection sans charge vérifie le code et les exemples et peut prendre plusieurs minutes. Elle exclut quatre anciens témoins actifs de limites ; elle n'exécute pas les essais réels cgroup. La suite complète standard conserve ces témoins et n'est pas la commande recommandée pour cette vérification. Certains tests peuvent être ignorés si leurs outils sont absents : lisez les éventuels `skipped`. Les essais séparés de `tests_real/` demandent un scope systemd utilisateur et ne font pas partie de la suite standard. Ils ne sont pas nécessaires pour une utilisation quotidienne.

Exemples utiles

```
python3 -m moulinette dashboard --no-open
python3 -m moulinette check ./exemples/c12/ex08 \
  --exercise c12/ex08 --experimental-sandbox --no-history
```

Pour consulter les détails de l'aide d'un diagnostic : par exemple `python3 -m moulinette sandbox doctor --help`.

08 · Dépannage

« No module named moulinette »

Vous n'êtes probablement pas dans le dossier du projet extrait, ou vous utilisez un autre environnement Python. Ouvrez un terminal à la racine et vérifiez la version :

```
cd ~/Téléchargements/moulinette-lab-v0.29-dev
python3 -c "import moulinette; print(moulinette.__version__)"
```

Adaptez le chemin si vous avez extrait l'archive ailleurs. N'installez pas une nouvelle version simplement pour corriger un mauvais dossier de travail.

Norminette absente ou résultat incomplet

Vérifiez `command -v norminette` puis `norminette --version`. La version attendue est 3.3.59 dans un environnement virtuel. Si vous l'aviez installée dans un environnement dédié, réactivez-le dans ce terminal. Une autre version ou une sortie illisible peut rendre le style indisponible.

Bubblewrap ou cgroup indisponible

Lisez le rapport et vérifiez la présence de `bwrap`. Un scope cgroup nécessite notamment un systemd utilisateur fonctionnel. En cas d'indisponibilité, la correction ne doit pas basculer silencieusement en mode local. Ne changez pas les protections système pour contourner l'erreur.

« Protection de pile » ou « Adaptation compilation »

Le moteur peut recompiler des objets avec `-fno-stack-protector` et les réanalyser en cas de symbole ambigu. Pour les projets compilés par le moteur, ces objets liés et testés n'ont donc plus cette protection. Pour un programme Makefile, le binaire d'origine reste testé : l'analyse d'objets recompilés séparément est **indicative** et ne prouve pas qu'ils sont ceux du binaire. Si l'ambiguïté persiste ou qu'une bibliothèque ne peut pas être vérifiée, le résultat reste incomplet.

La page ne s'ouvre pas ou le thème ne reste pas

Utilisez `dashboard --no-open`, puis ouvrez le chemin affiché dans votre navigateur. Vérifiez que vous utilisez le même navigateur et la même adresse de page, et que son stockage local n'est pas bloqué. La page signale si le stockage du thème est indisponible. Régénérez-la après chaque correction.

09 · Exercices et sujets

Versions des PDF fournis et nombre d'exercices intégrés :

- C00 v8 : 9 ; C01 v6 : 9 ; C02 v6 : 13.
- C03 v5 : 6 ; C04 v5 : 6 ; C05 v7 : 9.
- C06 v8 : 4 ; C07 v6 : 6 ; C08 v14 : 6.
- C09 v5.0 : 3 ; C10 v7.1 : 4 ; C11 v8.0 : 8.
- C12 v9.0 : 18 ; C13 v6.0 : 8. Total : **109**.

Les versions et numéros d'exercices des paires PDF/Markdown fournies concordent ; cela ne prouve pas une identité textuelle complète. Le Lab ne dispose pas des tests officiels 42.

Limites pédagogiques à connaître

- Les cas fonctionnels, erreurs d'allocation, comportements hors sujet et gestion mémoire ne sont pas tous testés. Aucun contrôle mémoire complet n'est fourni.
- C05 : la récursivité ou l'itération demandée n'est pas prouvée automatiquement. Pour les dix reines, les 724 solutions distinctes et leur validité sont contrôlées, pas la récursivité.
- C06 : la signature de `main` n'est pas vérifiée séparément ; un retour 0 est une convention locale.
- C10 : options, erreurs, grands fichiers et Makefiles ne sont contrôlés que partiellement. Les règles `all`, `clean` et `fclean` sont exercées, sans prouver toutes les dépendances ou suppressions.
- C11/ex05 : le tableau de pointeurs de fonctions demandé n'est pas démontré par les tests fonctionnels.
- C12 et C13 : les opérations de listes et d'arbres sont éprouvées sur des cas représentatifs, pas sur tous les cas possibles. Les sources C12 sont compilées en C11 pour les prototypes historiques du sujet.
- L'analyse des fonctions externes avec `nm` est partielle et ne constitue pas une garantie de sécurité ou de conformité.

En cas d'écart, consultez le sujet fourni et le diagnostic détaillé ; le Lab est une aide pour apprendre, pas l'autorité officielle.

10 · Sécurité, validation et suite

Utiliser uniquement des sources de confiance

La sécurité est en pause et n'est pas achevée. N'utilisez pas le Lab sur du code étudiant inconnu, un dépôt tiers non vérifié ou un programme hostile. Un Makefile ou un script de rendu peut exécuter des commandes. Bubblewrap, cgroup et les limites de capture réduisent certains risques mais ne constituent pas un confinement garanti. Le mode local n'isole rien.

Le témoin CPU/OOM dangereux de v0.20 a été désactivé depuis v0.21. **Ne l'exécutez pas dans l'ancienne v0.20**, et ne lancez pas de test de saturation sur votre session de travail. Les limites mémoire et CPU sous charge ne sont pas validées par les tests fonctionnels.

Ce qui a été testé

La v0.26.1 a réussi 296 tests standards sur l'Ubuntu de l'utilisateur ; la v0.27 a réussi 300 tests et la v0.28 304 tests dans l'environnement de développement. La refonte de page et ses thèmes ont également été testés dans Chromium, y compris avec le stockage bloqué. Cela ne démontre pas le comportement de tous les navigateurs ni la sécurité. La version courante peut nécessiter une nouvelle validation sur votre ordinateur.

Pour aller plus loin

- `README.md` : prise en main et limites de la version.
- `ETAT-CHANTIER.md` : état du développement et validations.
- `docs/feuille-de-route.md` : feuille de route et historique.
- `docs/` : notes détaillées et anciennes versions de guides. Consultez-les comme archives ; leurs indications de version peuvent être dépassées.
- `python3 -m moulinette ... --help` : référence des options disponibles dans la version installée.

Le présent PDF est un **manuel statique inclus dans le Lab**. Il n'inclut ni historique personnel ni correction individualisée. Vous pouvez consulter sa source éditable `docs/manuel-utilisateur.md`.